



資料分析與機器學習 (Data Analysis and Machine Learning)

Lecture 3 –

Adaline & NumPy, Pandas

教師：陳慶永

Outline

- **Adaptive Linear Neurons (Adaline)**
 - ▶ Activation function (激活函數)
 - ▶ Loss function (損失函數)
 - ▶ Gradient descent (梯度下降)
 - Stochastic gradient descent (SGD) (隨機梯度下降)
 - Mini-batch gradient descent
 - ▶ Feature scaling (特徵縮放)
- NumPy & Pandas
- Training an Adaline Model
- Classwork
 - ▶ Assignment #1



Illustrates the key concepts of defining and minimizing continuous loss functions



Adaptive Linear Neurons (Adaline)

Adaline

- **Adaline** (自適應線性神經元) algorithm (1960)
 - ▶ An improvement on PLA
 - Define and minimize a continuous **loss function** (損失函數)
 - Update weights based on a **linear activation function** (激活函數)
 - ▶ Lays the groundwork for understanding other ML algorithms (including multilayer neural networks)

Adaline

- Linear activation function

- ▶ Simply the identity function of the net input

$$\sigma(z) = z$$

- ▶ Used for learning the weights
- ▶ Still use a threshold function to make the final prediction

$$T(\sigma(z)) = \begin{cases} 1 & \sigma(z) \geq 0.5 \\ 0 & 0 \end{cases}$$

Adaline

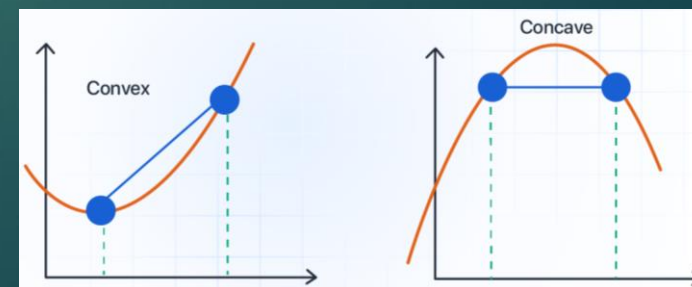
■ Loss function

- ▶ **Mean Squared Error (MSE)** (均方誤差) between the calculated outcomes and the true class labels

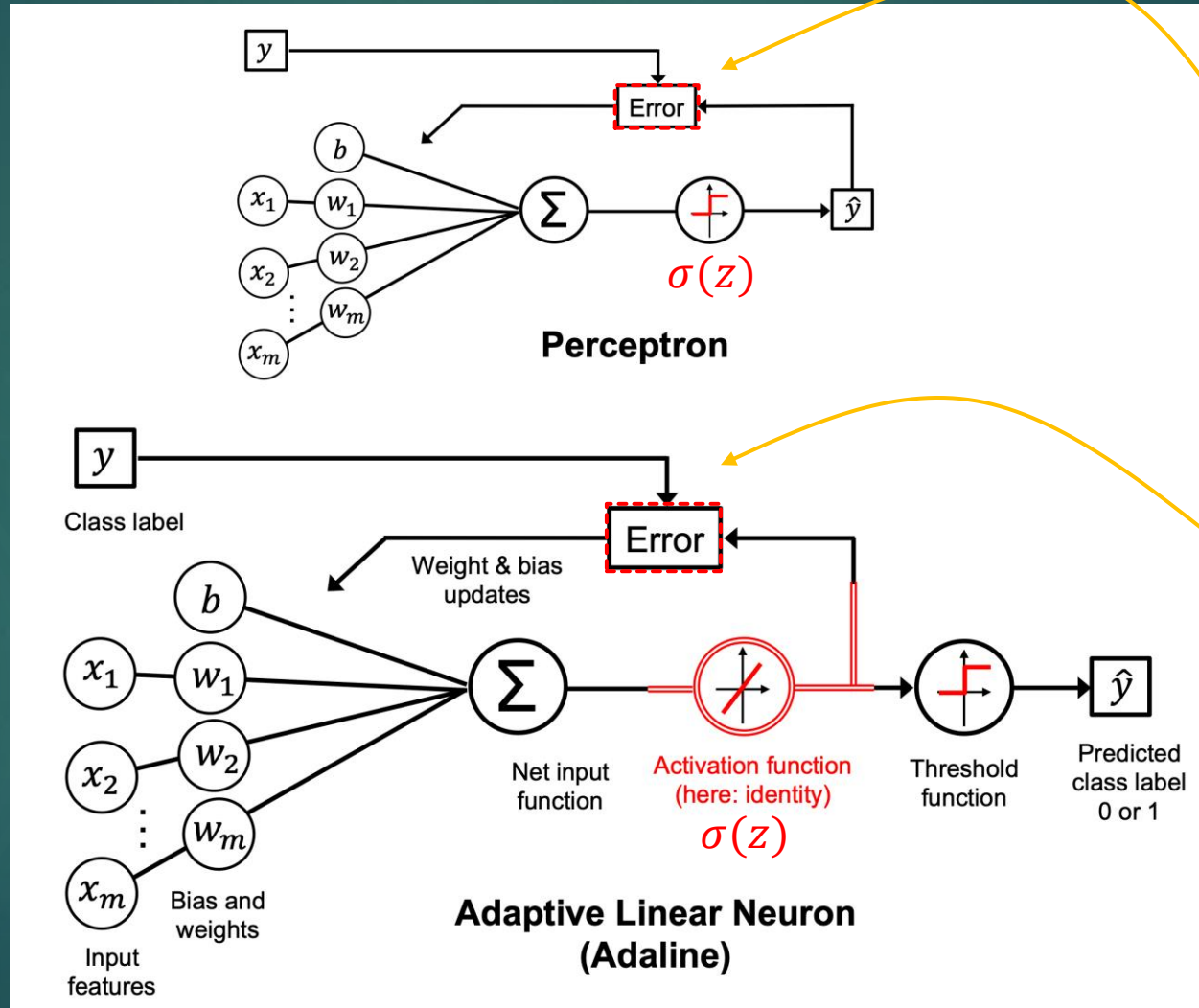
$$L(\mathbf{w}, b) = \frac{1}{\cancel{2}n} \sum_{i=1}^n \left(y^{(i)} - \sigma(z^{(i)}) \right)^2$$

(n is the number of total samples)

- Differentiable function (可微分的函數)
- Convex function (凸函數)



Perceptron v.s. Adaline



Compares the true class labels to the predicted class labels

Compares the true class labels with the linear activation function's continuous valued output

Gradient (梯度)

- The **gradient** (梯度) of a differentiable multivariate function $f(\mathbf{w})$ is a **vector** of its **partial derivatives**

$$\nabla f(\mathbf{w}) = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \\ \vdots \\ \frac{\partial f}{\partial w_m} \end{bmatrix}$$

- ▶ A vector that points in the direction of the *steepest ascent* of a scalar function f at a point (純量函數 f 在某一點增加最快的方向與變化率)

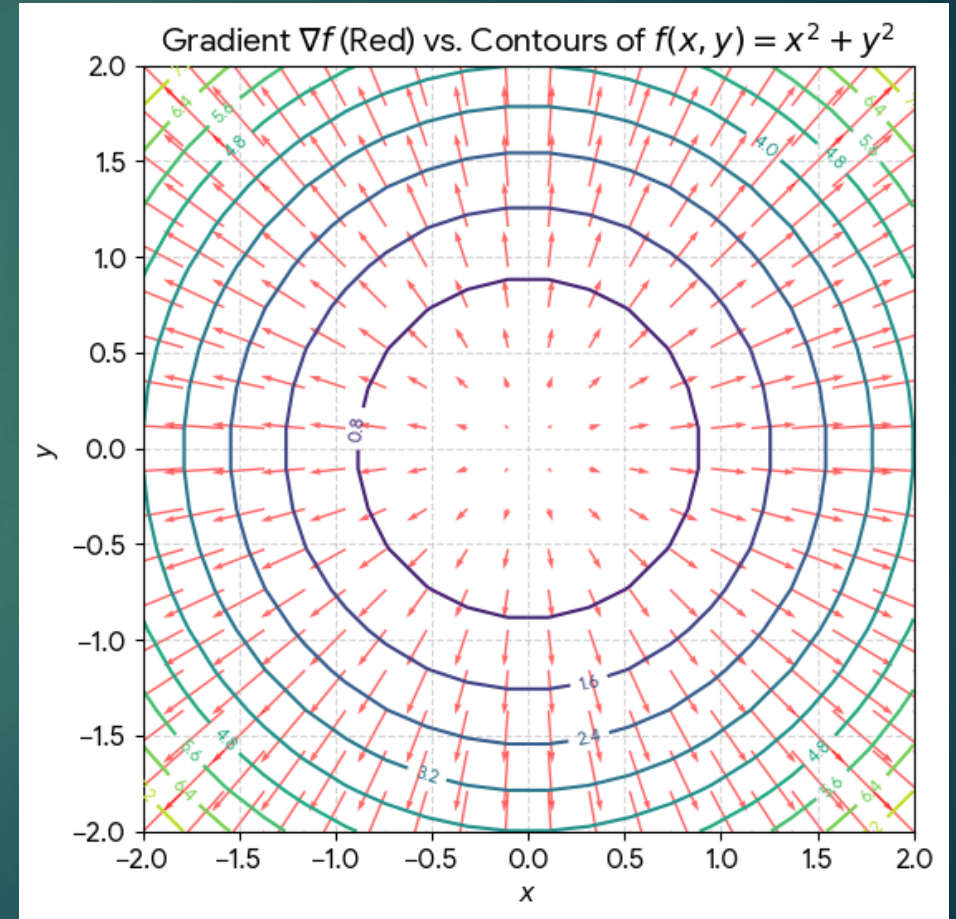
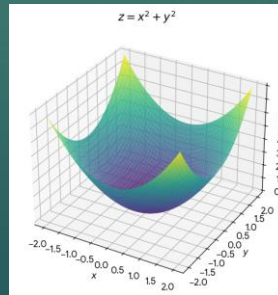
The opposite direction (negative gradient) is the direction of steepest descent.

Gradient (梯度)

■ Visualization

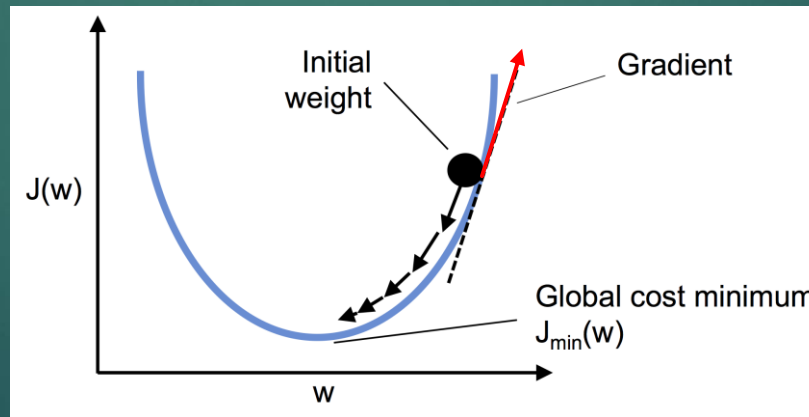
- ▶ $f(x, y) = x^2 + y^2$
 - Visualized by **contours**; or
 - Visualized by a **surface** in 3D space
- ▶ $\nabla f = [2x, 2y] = 2x\mathbf{i} + 2y\mathbf{j}$
 - Visualized by red **arrows**
 - Notice their lengths and directions

(Circular Paraboloid)



Gradient Descent (GD) (梯度下降)

- An **iterative algorithm** for **minimizing** a differentiable multivariate function
 - ▶ Take repeated *appropriate* steps in the *opposite* direction of the gradient (or approximate gradient) of the function at the current point



(For simplicity, we visualize the idea only for a single weight w)

Gradient Descent (GD) (梯度下降)

- Update model parameters

$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}, \quad b := b + \Delta b$$

$$\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L(\mathbf{w}, b), \quad \Delta b = -\eta \nabla_b L(\mathbf{w}, b)$$

(Negative gradient multiplied by the learning rate)

$$\Delta w_j = -\eta \frac{\partial L}{\partial w_j} \quad \text{and} \quad \Delta b = -\eta \frac{\partial L}{\partial b}$$

$$j = 1, 2, \dots, m$$

$$\frac{\partial L}{\partial w_j} = -\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)}$$

(Partial derivative of the loss function w.r.t. each weight)

$$\frac{\partial L}{\partial b} = -\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right)$$

(Partial derivative of the loss function w.r.t. the bias)

Gradient Descent (GD) (梯度下降)

■ Derivation of the partial derivative

$$\frac{d}{dx} \left(f(g(x)) \right) = f'(g(x)) g'(x)$$

$$\begin{aligned} \frac{\partial L}{\partial w_j} &= \frac{\partial}{\partial w_j} \frac{1}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right)^2 = \frac{1}{n} \frac{\partial}{\partial w_j} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right)^2 \\ &= \frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) \frac{\partial}{\partial w_j} \left(y^{(i)} - \sigma(z^{(i)}) \right) \text{ (Chain Rule)} \\ &= \frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) \frac{\partial}{\partial w_j} \left(y^{(i)} - \sum_k \left(w_k x_k^{(i)} + b \right) \right) \\ &= \frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) \left(-x_j^{(i)} \right) = \boxed{-\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)}} \end{aligned}$$

The same approach can be used to find partial derivative $\frac{\partial L}{\partial b}$.

Gradient Descent (GD) (梯度下降)

■ Notes

$$\frac{\partial L}{\partial w_j} = -\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)}$$

- ▶ Also referred to as *full batch gradient descent*
- ▶ The weight update is calculated based on *all* examples in the training dataset
 - In contrast, PLA updates the parameters incrementally after *each* training example

Gradient Descent (GD) (梯度下降)

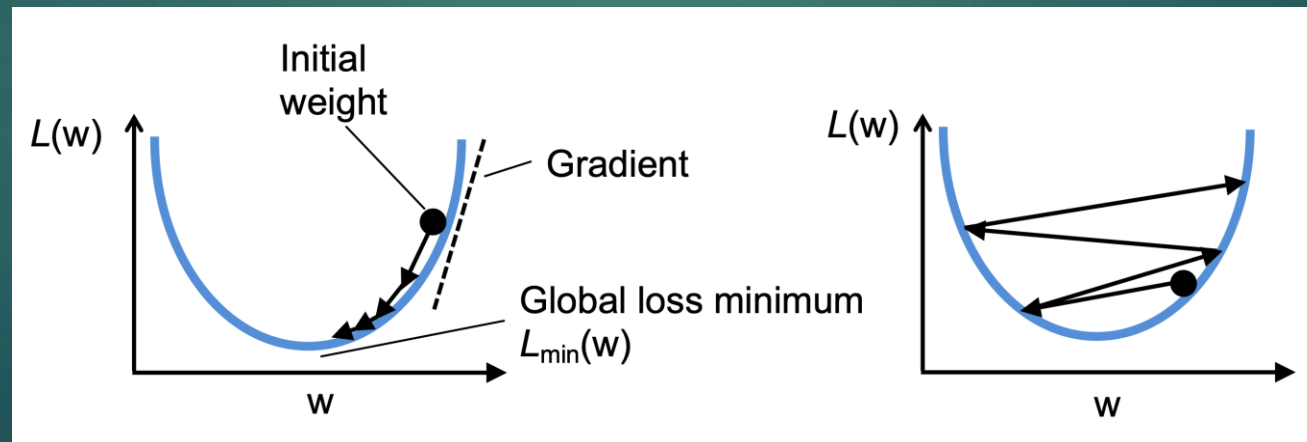
- Issue on **large-scale** dataset: millions of samples
 - ▶ Full batch GD is computationally quite costly
 - Gradient calculated from the *whole* training dataset each time we take one step update

Gradient Descent (GD) (梯度下降)

- Issue on **learning rate** η : Well-chosen v.s. too-large

$$\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L(\mathbf{w}, b), \quad \Delta b = -\eta \nabla_b L(\mathbf{w}, b)$$

- ▶ Well-chosen: the loss decreases gradually toward the global minimum
- ▶ Too-large: it overshoots the global minimum and even diverge



Hyperparameters (超參數)

- Learning rate (and number of epochs, etc.) are the so-called **hyperparameters** (or tuning parameters)
 - ▶ In practice, it requires some *experimentation* to find a good learning rate for *optimal* convergence
 - ▶ We will, in latter courses, take a look at different techniques to *automatically* find the values of different hyperparameters that yield optimal performance

Stochastic Gradient Descent (SGD)

$$\frac{\partial L}{\partial w_j} = -\frac{2}{n} \sum_i (y^{(i)} - \sigma(z^{(i)})) x_j^{(i)}$$

■ Stochastic GD (SGD) (隨機梯度下降)

- ▶ Update parameters incrementally for *each* training example
- ▶ Gradient is calculated based on a *single* training example

$$\Delta w_j = \eta (y^{(i)} - \sigma(z^{(i)})) x_j^{(i)}, \quad \Delta b = \eta (y^{(i)} - \sigma(z^{(i)}))$$

- Converges *faster* (than GD) because of more frequent weight updates
- Support **online learning** (trained on the fly as new training data arrives)
- ▶ It is *important* to present training data in a *random* order
 - **Shuffle** the training dataset for every epoch to *prevent cycles*

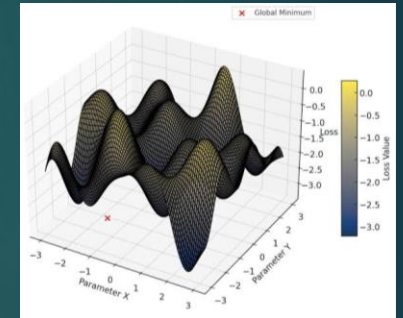
Stochastic Gradient Descent (SGD)

■ What happens w/o shuffling the training dataset?

核心問題	發生原因（機制）	帶來的後果
1. 陷入死循環 與規律震盪 (Cycling & Oscillation)	模型每個 Epoch 都以「完全相同的順序」讀取樣本，權重修正的路徑被規律化。	權重會在特定路徑上不斷 原地打轉 ，無法順利滑向全域最小值（Global Minimum）。
2. 序列偏見與 記憶效應 (Order Bias)	SGD 是看一筆更新一次。如果資料排列有隱含規律（例如相同類別集中），模型會被順序綁架。	模型在訓練後期會過度去適應最後看到的那些樣本， 遺忘前期的特徵分佈 。
3. 違反統計學 無偏估計 (Violation of i.i.d.)	失去隨機性後，單一樣本的梯度不再能代表整體母體的真實梯度方向。	算出來的梯度方向會 帶有強烈偏差 ，無法準確沿著代價函數的表面下降。

Stochastic Gradient Descent (SGD)

- Error surface is noisier than in GD
 - ▶ SGD *can* escape shallow local minima of nonlinear loss functions
- Adjusting the learning rate during training
 - ▶ In SGD implementations, learning rate is often replaced by an **adaptive learning rate** that *decreases over time*



$$\frac{c_1}{[\text{number of iterations}] + c_2}$$

Mini-batch Gradient Descent

■ Mini-batch GD

- ▶ A compromise between full batch GD and SGD
 - Can be understood as applying full batch GD to *smaller subsets* of the training data
- ▶ Maximizes computational efficiency while stabilizing the learning process
 - Replace the for loop over the training examples in SGD with vectorized operations
 - **Batch size** is a crucial hyperparameter
 - Standard sizes are powers of 2 (e.g., 32, 64, 128)

Feature Scaling (特徵縮放)

- Original features (variables) may have *different* ranges of distribution
 - ▶ $[0,1]$, $[-1,1]$, $[0, 255]$, $(-\infty, \infty)$, ...
 - ▶ Features with larger numeric values *would* dominate the learning process
- Many ML algorithms (relying on *distance* calculations or *gradient*) require some sort of **feature scaling** for optimal performance
 - ▶ GD is one of them

Feature Scaling (特徵縮放)

- Feature scaling is a **data preprocessing** (資料前處理) technique to *normalize* the range of features
 - ▶ Brings all features to the same level
- **Standardization** (標準化) (one of the common feature scaling methods)
 - ▶ Shifts the mean (平均值) of *each* feature to zero
 - ▶ *Each* feature has a standard deviation (std) (標準差) of 1 (unit variance)

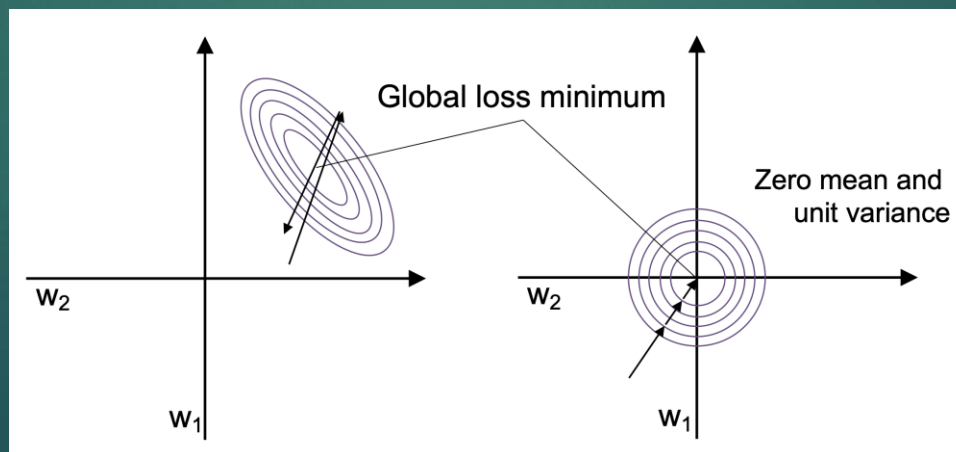
$$x'_j = \frac{x_j - \mu_j}{\sigma_j} \quad j = 1, 2, \dots, m$$

Standardization does *not* make the original dataset normally distributed.

Feature Scaling (特徵縮放)

- Feature scaling helps balanced and stable model training
 - ▶ Easier to find a learning rate that works well for all weights
 - If the features are on vastly different scales, a learning rate that works well for updating one weight might be too large or too small to update other weights equally well

(同一個學習率無法同時適合不同尺度特徵的權重更新)



(同一個學習率適合相同尺度特徵的權重更新)

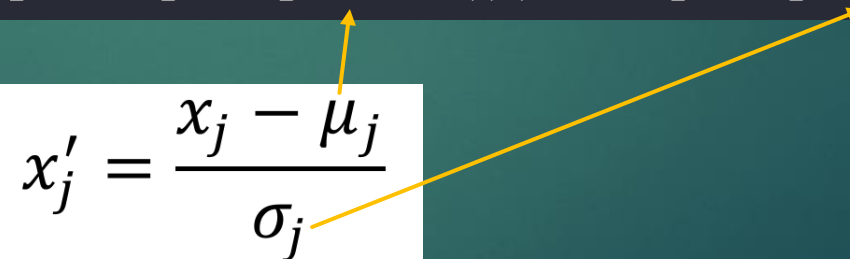
Standardization in Python

■ Standardization via the NumPy methods

▶ `mean()`

▶ `std()`

```
X_std = np.copy(X)
X_std[:,0] = (X[:,0] - X[:,0].mean()) / X[:,0].std()
X_std[:,1] = (X[:,1] - X[:,1].mean()) / X[:,1].std()
```

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$


NumPy



NumPy Arrays (**ndarray**)

- **NumPy** is probably the most fundamental numerical computing module in Python
- We will use **array/matrix** a lot later
 - ▶ [numpy.ndarray's documentation](#) (Check it out)
 - ▶ [Broadcasting](#)
 - describes how NumPy treats **arrays with different shapes** during arithmetic operations
- See Part II (Introduction to NumPy) of the Textbook for more details

Arrays as Vectors and Matrices

- NumPy is the fundamental library for linear algebra
 - ▶ While SciPy provides advanced algorithms (via `scipy.linalg`), the data structures themselves—vectors and matrices—are almost always handled as NumPy Arrays (`ndarray`)
 - Vector: 1D array
 - Matrix: 2D array

Arrays as Vectors and Matrices

■ Common Functions and Attributes

Function / Attribute	Description
<code>np.array()</code>	Creates an array from a Python list.
<code>np.zeros(shape)</code>	Creates an array filled with 0s.
<code>np.ones(shape)</code>	Creates an array filled with 1s.
<code>np.eye(n)</code>	Creates an Identity Matrix .
<code>array.ndim</code>	Returns the dimension (e.g., 2 for a matrix).
<code>array.shape</code>	Returns the shape (e.g., (rows, cols)).
<code>array.dtype</code>	Returns the data type of the elements (e.g., float64, int32).

Dimension and Shape

- Dimension (維度): `arr.ndim`
- Shape (形狀): `arr.shape`

Array Content	ndim	shape	Interpretation
<code>[1, 2, 3]</code>	1	<code>(3,)</code>	1D array with 3 elements.
<code>[[1, 2], [3, 4]]</code>	2	<code>(2, 2)</code>	2D array with 2 rows and 2 columns.
<code>[[[1], [2]]]</code>	3	<code>(1, 2, 1)</code>	3D array with layers/rows/cols of 1x2x1.

NOTE: A shape of `(3,)` is NOT the same as `(3, 1)` or `(1, 3)`. The former is a rank-1 array (sequence), while the latter two are rank-2 matrices (explicit rows/columns).

Arrays as Vectors and Matrices

■ Notes

- ▶ A vector (1D array) with **shape (3,)** is neither a row nor a column vector by default
- ▶ If you specifically need a **Column Vector** for certain matrix operations, you should define it with **shape (3, 1)** using `np.array([[1], [2], [3]])`

Arrays as Vectors and Matrices

■ Example 1

▶ Sample code: 03_1.py

```
import numpy as np
from scipy import linalg

# 1. 定義向量 (Vector)
# 一維陣列通常被視為列向量或行向量的基礎
vec = np.array([1, 2, 3])
print("向量內容:", vec)
print("向量維度 (Dimension):", vec.ndim) # 輸出 1
print("向量形狀 (Shape):", vec.shape) # 輸出 (3,)

# 2. 定義矩陣 (Matrix)
# 使用巢狀列表建立 2x3 矩陣
mat = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print("\n2x3 矩陣:\n", mat)
print("矩陣維度 (Dimension):", mat.ndim) # 輸出 2
print("矩陣形狀 (Shape):", mat.shape) # 輸出 (2, 3)
```

Arrays as Vectors and Matrices

3. 特殊矩陣建立

```
zeros_mat = np.zeros((2, 2))      # 2x2 全零矩陣
ones_mat = np.ones((3, 2))        # 3x2 全一矩陣
identity_mat = np.eye(3)          # 3x3 單位矩陣 (對角線為 1)
print("\n單位矩陣:\n", identity_mat)
```

4. 指定資料型別 (Data Types)

預設通常是浮點數或根據輸入判定，也可以手動指定

➡

```
float_vec = np.array([1, 2, 3], dtype='float64')
print("\n浮點數向量型別:", float_vec.dtype) # 輸出 float64
```

5. --- SciPy Note ---

SciPy functions usually take NumPy arrays as input.

For example, checking the norm (magnitude) of a vector:

```
v_norm = linalg.norm(vec)
print(f"\nNorm of the vector: {v_norm:.2f}") # 輸出 3.74
```

Operations

- `numpy.linalg` is the primary tool for addition, scalar multiplication, and matrix multiplication
 - ▶ Basic arithmetic is performed element-wise
 - Addition (+)
 - Scalar multiplication (*)
 - ▶ **Matrix multiplication** requires specific functions or operators
 - @ (= `np.matmul()`)
 - `np.dot()`

```
numpy.dot(a, b, out=None)
```

Dot product of two arrays. Specifically,

- If both *a* and *b* are 1-D arrays, it is inner product of vectors (without complex conjugation).
- If both *a* and *b* are 2-D arrays, it is matrix multiplication, but using `matmul` or `a @ b` is preferred.
- If either *a* or *b* is 0-D (scalar), it is equivalent to `multiply` and using `numpy.multiply(a, b)` or `a * b` is preferred.

Operations

■ Common Functions and Operators

Operation Type	Operator / Function	Description
Addition	+ or <code>np.add()</code>	Adds corresponding elements of two arrays of the same shape.
Scalar Mult.	*	Multiplies every element in the array by a single number.
Transpose	.T	View of the transposed array. (no effect on 1D array)
Dot Product	<code>np.dot()</code>	Calculates the inner product of two vectors.
Matrix Mult.	@ (= <code>np.matmul()</code>) or <code>np.dot()</code>	Performs standard matrix multiplication.

Operations

■ np.dot() vs. np.matmul()

▶ For 2D Matrices: They are identical

- np.dot(A, B)
- np.matmul(A, B)
- A @ B

▶ For Higher Dimensions (3D+ Tensor): np.matmul (or @) is preferred

- np.matmul() handles "batches" of matrices (broadcasting) correctly, which is the standard in Deep Learning
- (10, 3, 4) @ (4, 5) → (10, 3, 5)

Operations

■ Example 2

▶ Sample code:
03_2.py

```
[[ 6  8]
 [10 12]]
```

```
[[10 20]
 [30 40]]
```

```
import numpy as np

# Define two 2x2 matrices
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# --- 1. Matrix Addition ---
# Condition: Dimensions must be the same
# OR compatible for Broadcasting.
add_result = A + B
print("Matrix Addition (A + B):\n", add_result)

# --- 2. Scalar Multiplication ---
# Multiplies every element by the constant.
scalar = 10
scalar_result = A * scalar
print(f"\nScalar Multiplication ({scalar} * A):\n", scalar_result)
```

Operations

```
# --- 3. Element-wise Multiplication (Caution!) ---
# This is NOT standard matrix multiplication. It multiplies position by position.
element_wise = A * B
print("\nElement-wise Multiplication (A * B):\n", element_wise)
```

```
[[ 5 12]
 [21 32]]
```

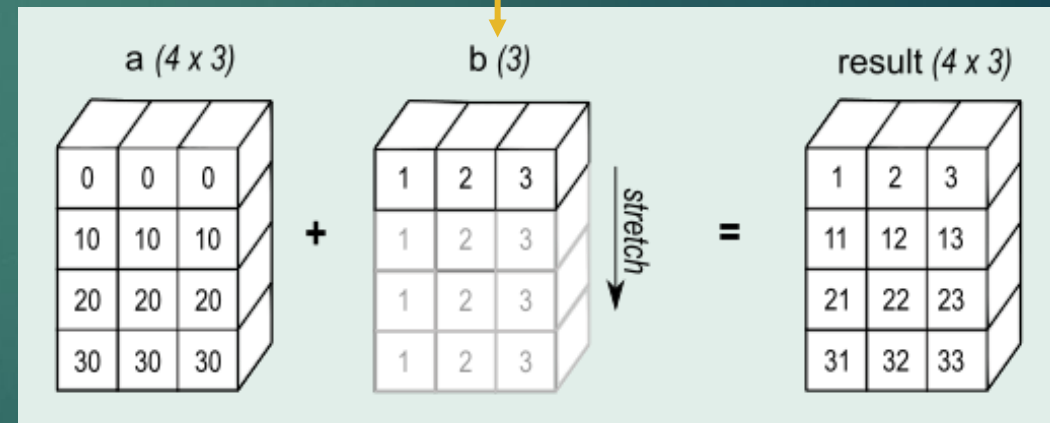
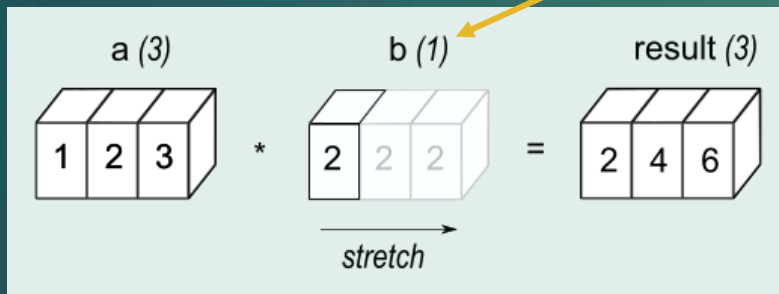
```
# --- 4. Standard Matrix Multiplication ---
# This follows the linear algebra rule (rows of A times columns of B).
# Condition: Number of columns in A must equal number of rows in B.
C = np.array([[1, 0, 1], [0, 1, 0]]) # 2x3 Matrix
matrix_mult = A @ C # Preferred modern syntax
# Alternative: np.matmul(A, C) or np.dot(A, C)
print("\nMatrix Multiplication (A @ C, results in 2x3):\n", matrix_mult)
```

```
[[1 2 1]
 [3 4 3]]
```

```
# --- 5. Vector Dot Product ---
v = np.array([1, 2, 3])
w = np.array([4, 5, 6])
dot_product = np.dot(v, w) # 1*4 + 2*5 + 3*6 = 32
print(f"\nVector Dot Product (v · w): {dot_product}")
```

NumPy Broadcasting

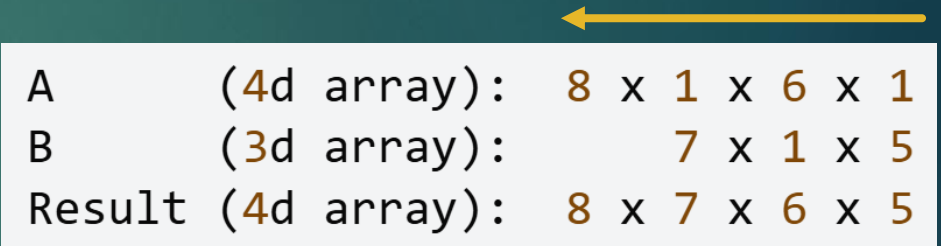
- Subject to certain constraints, the smaller array is “broadcast” across the larger array so that they have compatible shapes



General Broadcasting Rules

- When operating on two arrays, NumPy compares their shapes element-wise. It starts with the trailing (i.e. rightmost) dimension and works its way left. Two dimensions are **compatible** when

- ▶ they are equal, or
- ▶ one of them is 1



A	(4d array):	8	x	1	x	6	x	1
B	(3d array):			7	x	1	x	5
Result	(4d array):	8	x	7	x	6	x	5

- If these conditions are not met, a *ValueError* exception is thrown, indicating that the arrays have incompatible shapes

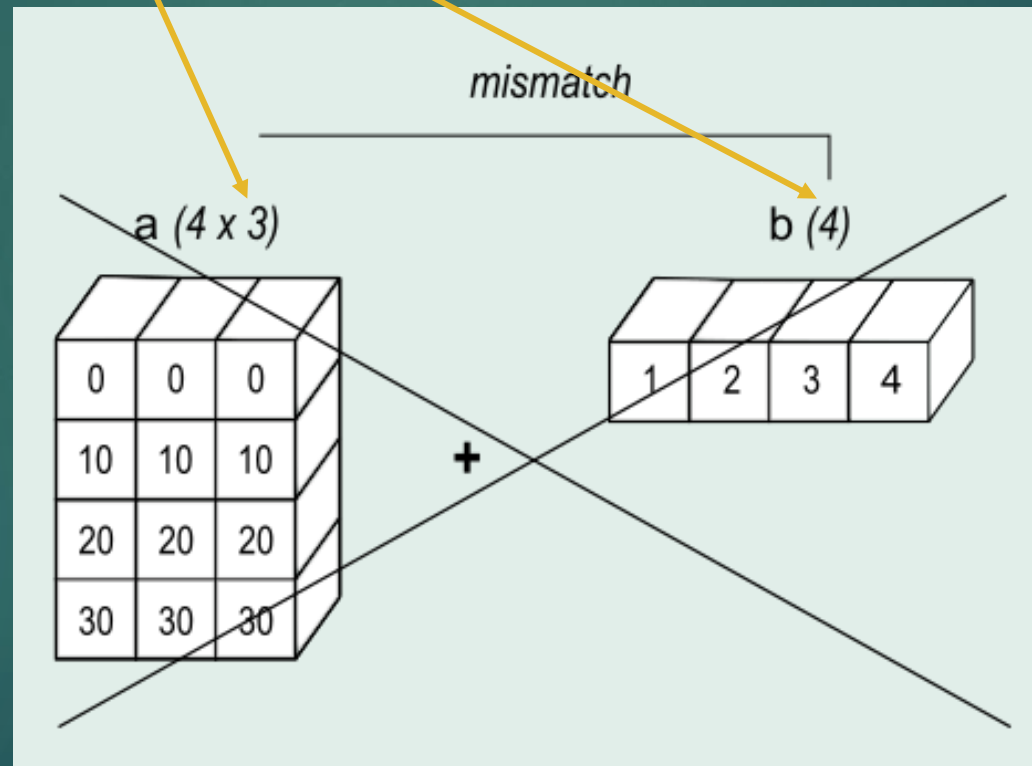
General Broadcasting Rules

■ Notes:

- ▶ Input arrays do not need to have the same number of dimensions
 - The resulting array will have the same number of dimensions as the input array with the greatest number of dimensions
 - The size of each dimension is the largest size of the corresponding dimension among the input arrays
- ▶ Missing dimensions are assumed to have size **one**

NumPy Broadcasting

- Incompatible ($3 \neq 4$)



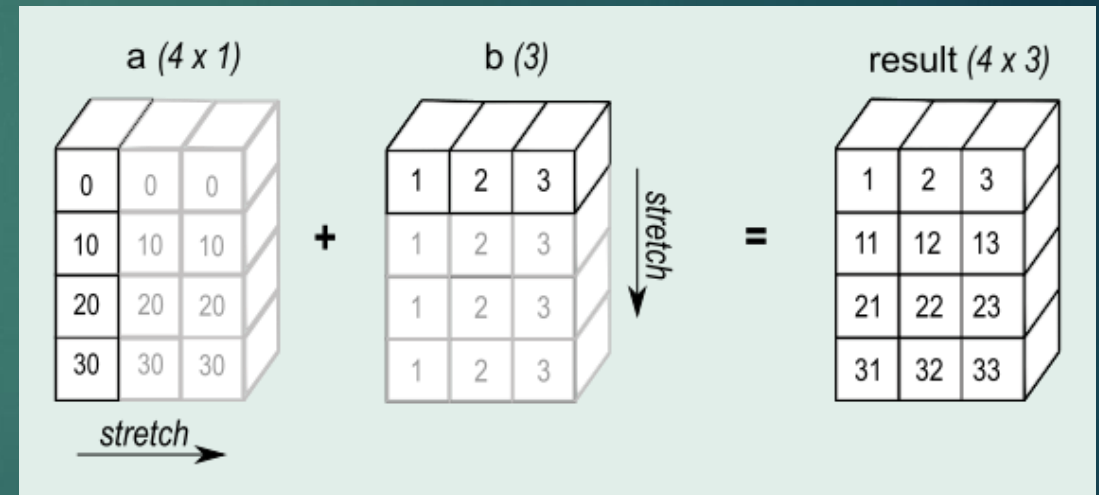
NumPy Broadcasting

■ np.newaxis

- ▶ an **index operator** used to **insert a new axis** to an array

```
>>> import numpy as np
>>> a = np.array([0.0, 10.0, 20.0, 30.0])
>>> b = np.array([1.0, 2.0, 3.0])
>>> a[:, np.newaxis] + b
array([[ 1.,  2.,  3.],
       [11., 12., 13.],
       [21., 22., 23.],
       [31., 32., 33.]])
```

**a.shape = (4,)
→ (4,1)**



NumPy Broadcasting

- 題目：給定兩組二維座標點集 $A(3,2)$ 與 $B(5,2)$
 - ▶ 請利用 **Broadcasting** 計算清單 A 中每一點到清單 B 中每一點的歐幾里得距離 (Euclidean Distance or L2 norm)
 - ▶ 輸出應為一個 $\text{shape} = (3, 5)$ 的**距離矩陣**

Matrix A:

```
[[0 1]
 [2 3]
 [4 5]]
```

Matrix B:

```
[[0 1]
 [2 3]
 [4 5]
 [6 7]
 [8 9]]
```

Distance Matrix of Shape (3, 5):

```
[[ 0.          2.82842712  5.65685425  8.48528137 11.3137085 ]
 [ 2.82842712  0.          2.82842712  5.65685425  8.48528137]
 [ 5.65685425  2.82842712  0.          2.82842712  5.65685425]]
```

- 複習重點：np.newaxis, np.linalg.norm()

NumPy Broadcasting

■ 參考解答：03_3.py

```
import numpy as np
A = np.arange(6).reshape(3,2)
B = np.arange(10).reshape(5,2)
# (3, 1, 2) - (5, 2) -> (3, 5, 2)
→ dist_matrix = np.linalg.norm(A[:, np.newaxis, :] - B, axis=2)

print(f"Matrix A:\n{A}")
print(f"Matrix B:\n{B}")
print(f"Distance Matrix of Shape {dist_matrix.shape}:\n{dist_matrix}")
```

Pandas



Pandas

- Pandas is a library providing easy-to-use **data structures** and **data analysis** tools for Python
- Two primary data structures

Dimensions	Name	Description
1	Series	1D labeled homogeneously-typed array
2	DataFrame	General 2D labeled, size-mutable tabular structure with potentially heterogeneously-typed column

- See Part III (Data Manipulation with Pandas) of the Textbook for more details

Series

- 1D array of indexed data
 - ▶ Constructing Series objects

```
class pandas.Series(data=None, index=None, dtype=None, name=None, copy=None)
```

- data: data to be stored (can be a list, dict, ndarray or scalar)
- index: a list of axis labels
- Example:

```
➔ data = pd.Series([0.25, 0.5, 0.75, 1.0],  
                    index=['a', 'b', 'c', 'd'])
```

```
data['b']  
0.5
```

(as a generalized NumPy array)

a	0.25
b	0.50
c	0.75
d	1.00

Series

- Series combines a sequence of values with an explicit sequence of indices
 - ▶ Use the Series.to_numpy() method to convert to a NumPy ndarray (of values in Series)
 - Example:

```
# Create a sample Series
s = pd.Series([10, 20, 30, 40])

# Convert to NumPy array
→ arr = s.to_numpy()

print(arr)
# Output: [10 20 30 40]
```

DataFrame

- DataFrame is an analog of a 2D array with explicit row and column indices
 - ▶ Think of a DataFrame as a sequence of *aligned* Series
 - ▶ Constructing DataFrame objects

```
class pandas.DataFrame(data=None, index=None, columns=None,  
dtype=None, copy=None)
```

- *data*: data to be stored (can be a Series, dict, list of dicts or ndarray)
- *index*: a list of *row* labels
- *columns*: a list of *column* labels

DataFrame

- Example:

```
pd.DataFrame(np.random.rand(3, 2),
             columns=['foo', 'bar'],
             index=['a', 'b', 'c'])
```

	foo	bar
a	0.471098	0.317396
b	0.614766	0.305971
c	0.533596	0.512377

► Use the DataFrame.`to_numpy()` method to convert to a NumPy ndarray

- Example:

```
# Create a sample DataFrame
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})

# Convert the DataFrame to a NumPy array
→ arr = df.to_numpy()
```

```
print(arr)
# Output:
# [[1 3]
#  [2 4]]
```



Training an Adaline Model

Mission

- Implement the Adaline in Python
 - ▶ The PLA and Adaline are very similar
 - ▶ Simple modifications to the perceptron implementation
 - Weight and bias parameters are now updated by minimizing the MSE loss function
 - (Full batch) GD v.s. SGD
 - With v.s. without feature scaling
- Apply it to the Iris dataset
- Sample code: `03_4.py`

GD w/o Feature Scaling

■ AdalineGD

```
7 # 1.
8 # Implementing an adaptive linear neuron (Adaline)
9 # Optimization algorithm: Gradient Descent (GD)
10 class AdalineGD:
11     """ADaptive LInear NEuron classifier.
12
13     Parameters
14     -----
15     eta : float
16         Learning rate (between 0.0 and 1.0)
17     n_iter : int
18         Passes over the training dataset.
19     random_state : int
20         Random number generator seed for random weight
21         initialization.
22
23     Attributes
24     -----
25     w_ : 1d-array
26         Weights after fitting.
27     b_ : Scalar
28         Bias unit after fitting.
29     losses_ : list
30         Mean squared error loss function values in each epoch.
31
32     """
```

```
33 > def __init__(self, eta=0.01, n_iter=50, random_state=1): ...
37 >
38 > def fit(self, X, y): ...
80 >
81 > def net_input(self, X): ...
84 >
85 > def activation(self, X): ...
88 >
89 > def predict(self, X): ...
```

```
33 def __init__(self, eta=0.01, n_iter=50, random_state=1):
34     self.eta = eta
35     self.n_iter = n_iter
36     self.random_state = random_state
```

GD w/o Feature Scaling

```
38 def fit(self, X, y):
39     """ Fit training data.
40
41     Parameters
42     -----
43     X : {array-like}, shape = [n_examples, n_features]
44         Training vectors, where n_examples is the number of examples and
45         n_features is the number of features.
46     y : array-like, shape = [n_examples]
47         Target values.
48
49     Returns
50     -----
51     self : object
52
53     """
54     rgen = np.random.RandomState(self.random_state)
55     self.w_ = rgen.normal(loc=0.0, scale=0.01, size=X.shape[1])
56     self.b_ = np.float64(0.)
57     self.losses_ = []
```

(Initialization of weights and bias unit)

```
59 for i in range(self.n_iter):
60     net_input = self.net_input(X)
61     # Please note that the "activation" method has no effect
62     # in the code since it is simply an identity function. We
63     # could write `output = self.net_input(X)` directly instead.
64     # The purpose of the activation is more conceptual, i.e.,
65     # in the case of logistic regression (as we will see later),
66     # we could change it to
67     # a sigmoid function to implement a logistic regression classifier.
68     output = self.activation(net_input)
69     errors = (y - output)
70
71     #for w_j in range(self.w_.shape[0]):
72     #    self.w_[w_j] += self.eta * (2.0 * (X[:, w_j]*errors)).mean()
73
74     #self.w_ += self.eta * 2.0 * X.T.dot(errors) / X.shape[0]
75     self.w_ += self.eta * 2.0 * (X.T @ errors) / X.shape[0]
76     self.b_ += self.eta * 2.0 * errors.mean()
77     loss = (errors**2).mean()
78     self.losses_.append(loss)
79 return self
```

(Update parameters by GD,
see p. 11)

$$\frac{\partial L}{\partial w_j} = -\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)}$$

GD w/o Feature Scaling

```
81 ✓ def net_input(self, X):  
82     """Calculate net input"""  
83     return X @ self.w_ + self.b_  
84  
85 ✓ def activation(self, X):  
86     """Compute linear activation"""  
87     return X  
88  
89 ✓ def predict(self, X):  
90     """Return class label after unit step"""  
91     return (self.activation(self.net_input(X)) >= 0.5).astype(int)
```

(Linear activation function, see p. 5)

(Threshold function to make the final prediction, see p. 5)

GD w/o Feature Scaling

■ Load the Iris Dataset

```
94 # 2.
95 # Reading-in the Iris data
96 s = 'iris.data'
97 df = pd.read_csv(s, header=None, encoding='utf-8')
98
99 # Target array: select 2 classes only
100 # setosa (class '0') and versicolor (class '1')
101 y = df.iloc[0:100, 4].to_numpy()
102 y = np.where(y == 'Iris-setosa', 0, 1)
103
104 # Feature matrix: select 2 features only
105 # sepal length and petal length
106 X = df.iloc[0:100, [0, 2]].to_numpy()
```

GD w/o Feature Scaling

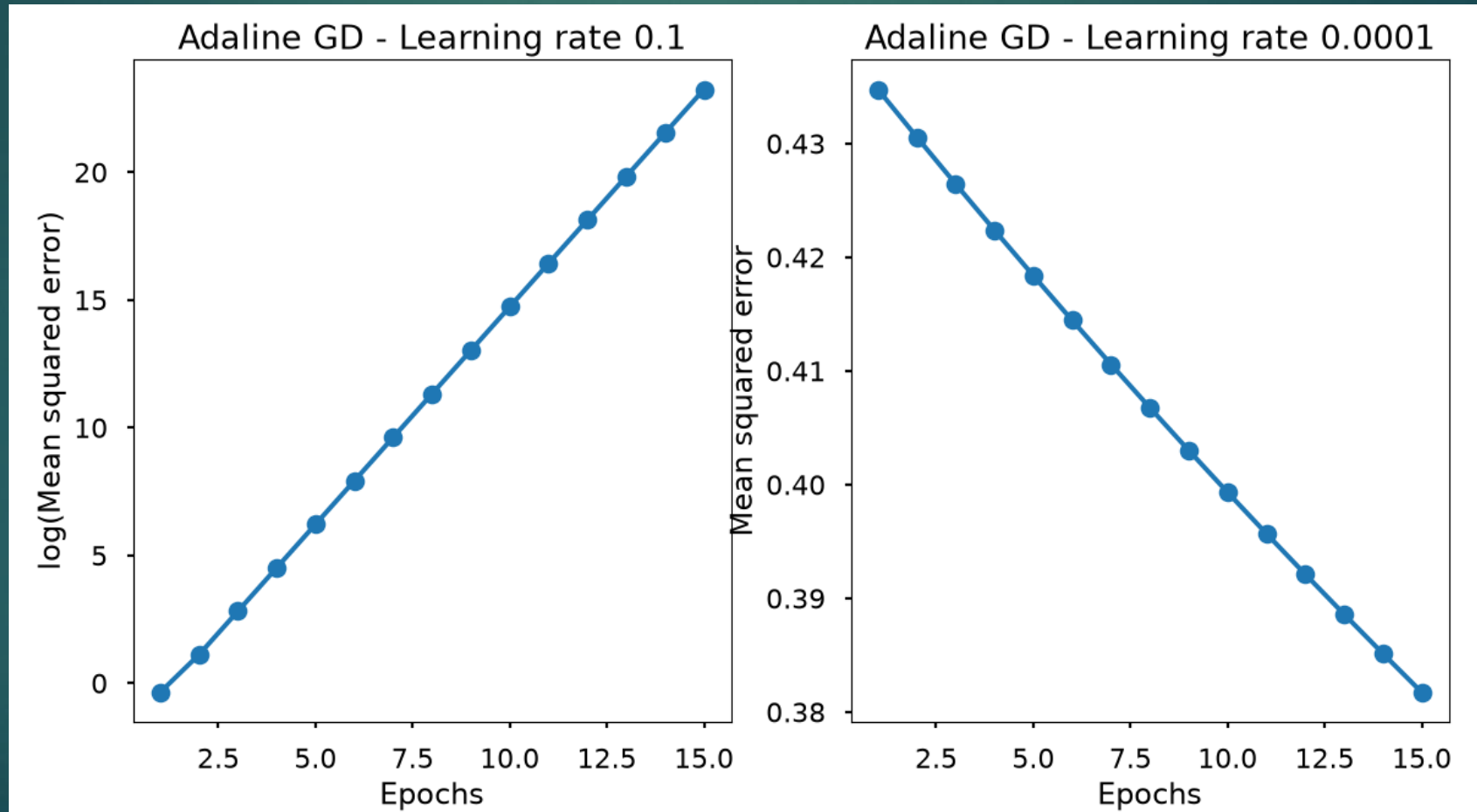
- Train the Adaline model & plot the **loss curve** (損失曲線)

▶ $\eta = 0.1$ v.s. $\eta = 0.0001$

```
109 # 3.
110 # Training the Adaline model on the Iris dataset
111 fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))
112 # Too-large learning rate (eta)
113 → ada1 = AdalineGD(n_iter=15, eta=0.1).fit(X, y)
114 ax[0].plot(range(1, len(ada1.losses_) + 1), np.log10(ada1.losses_), marker='o')
115 ax[0].set_xlabel('Epochs')
116 ax[0].set_ylabel('log(Mean squared error)')
117 ax[0].set_title('Adaline GD - Learning rate 0.1')
118 # Too-small learning rate (eta)
119 → ada2 = AdalineGD(n_iter=15, eta=0.0001).fit(X, y)
120 ax[1].plot(range(1, len(ada2.losses_) + 1), ada2.losses_, marker='o')
121 ax[1].set_xlabel('Epochs')
122 ax[1].set_ylabel('Mean squared error')
123 ax[1].set_title('Adaline GD - Learning rate 0.0001')
124 plt.show()
```

GD w/o Feature Scaling

- Loss curve (also called training curve)



GD w/o Feature Scaling

■ Two different types of convergence problems 🥲

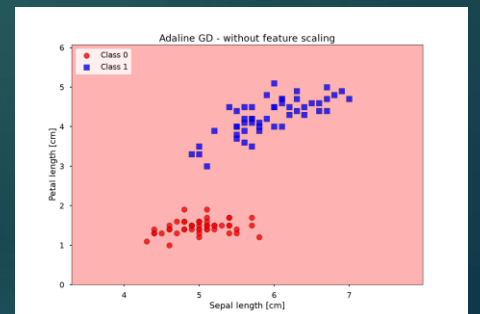
▶ Too-large learning rate

- The loss (MSE) becomes larger in every epoch 🤯
- Overshoot the global minimum (diverge)

▶ Too-small learning rate

- The loss (MSE) decreases slowly 🤔
- Require a very large number of epochs to converge

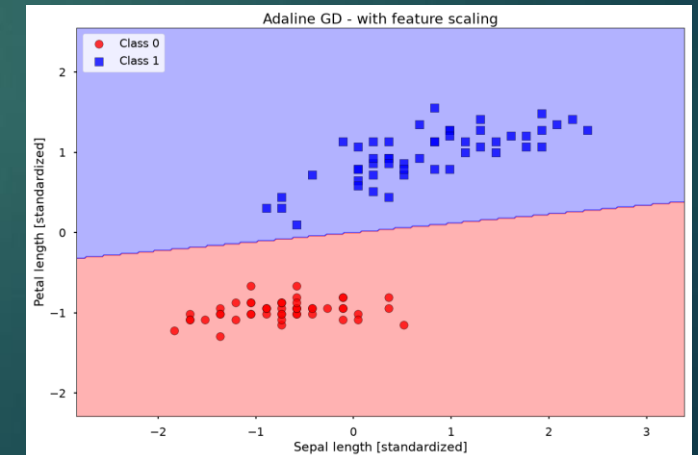
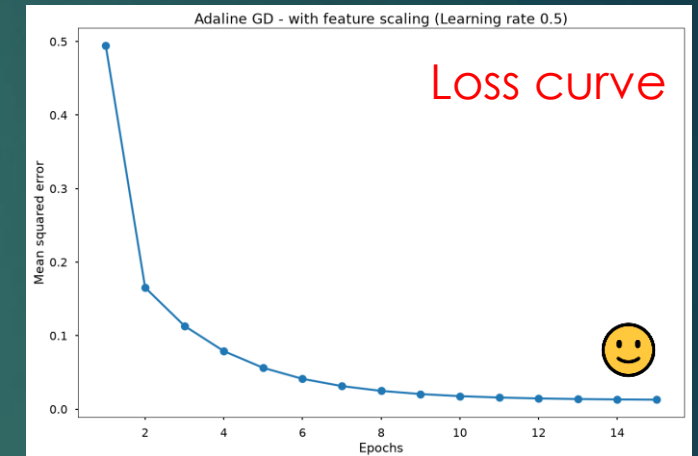
(Fail to separate setosa from versicolor flowers)



GD with Feature Scaling

■ Feature scaling using standardization

```
165 # 5. Improving GD through feature scaling
166 # Standardization: zero-mean and unit-std
167 # standardize features
168 X_std = np.copy(X)
169 X_std[:, 0] = (X[:, 0] - X[:, 0].mean()) / X[:, 0].std()
170 X_std[:, 1] = (X[:, 1] - X[:, 1].mean()) / X[:, 1].std()
171
172 ada_gd = AdalineGD(n_iter=15, eta=0.5)
173 ada_gd.fit(X_std, y)
174 plt.plot(range(1, len(ada_gd.losses_) + 1), ada_gd.losses_, marker='o')
175 plt.title('Adaline GD - with feature scaling (Learning rate 0.5)')
176 plt.xlabel('Epochs')
177 plt.ylabel('Mean squared error')
178 plt.tight_layout()
179 plt.show()
```



SGD with Feature Scaling

■ AdalineSGD

```
190 # 6. Adaline SGD
191 # Optimization algorithm: Stochastic Gradient Descent (SGD)
192 class AdalineSGD:
193     """ADaptive LInear NEuron classifier.
194
195     Parameters
196     -----
197     eta : float
198         Learning rate (between 0.0 and 1.0)
199     n_iter : int
200         Passes over the training dataset.
201     shuffle : bool (default: True)
202         Shuffles training data every epoch if True to prevent cycles.
203     random_state : int
204         Random number generator seed for random weight
205         initialization.
206
207     Attributes
208     -----
209     w_ : 1d-array
210         Weights after fitting.
211     b_ : Scalar
212         Bias unit after fitting.
213     losses_ : list
214         Mean squared error loss function value averaged over all
215         training examples in each epoch.
216
217     """
218
```

```
219 > def __init__(self, eta=0.01, n_iter=10, shuffle=True, random_state=None): ...
225
226 > def fit(self, X, y): ...
253
254 > def partial_fit(self, X, y): ...
264
265 > def _shuffle(self, X, y): ...
269
270 > def _initialize_weights(self, m): ...
276
277 > def _update_weights(self, xi, target): ...
285
286 > def net_input(self, X): ...
289
290 > def activation(self, X): ...
293
294 > def predict(self, X): ...
```

```
219 def __init__(self, eta=0.01, n_iter=10, shuffle=True, random_state=None):
220     self.eta = eta
221     self.n_iter = n_iter
222     self.w_initialized = False
223     self.shuffle = shuffle
224     self.random_state = random_state
```

SGD with Feature Scaling

```
226 ✓ def fit(self, X, y):
227 ✓     """ Fit training data.
228
229     Parameters
230     -----
231     X : {array-like}, shape = [n_examples, n_features]
232         Training vectors, where n_examples is the number of examples and
233         n_features is the number of features.
234     y : array-like, shape = [n_examples]
235         Target values.
236
237     Returns
238     -----
239     self : object
240
241     """
242     self._initialize_weights(X.shape[1])
243     self.losses_ = []
244     for i in range(self.n_iter):
245         if self.shuffle:
246             X, y = self._shuffle(X, y)
247             losses = []
248             for xi, target in zip(X, y):
249                 losses.append(self._update_weights(xi, target))
250             avg_loss = np.mean(losses)
251             self.losses_.append(avg_loss)
252     return self
```

```
265 def _shuffle(self, X, y):
266     """Shuffle training data"""
267     r = self.rgen.permutation(len(y))
268     return X[r], y[r]
269
270 def _initialize_weights(self, m):
271     """Initialize weights to small random numbers"""
272     self.rgen = np.random.RandomState(self.random_state)
273     self.w_ = self.rgen.normal(loc=0.0, scale=0.01, size=m)
274     self.b_ = np.float64(0.)
275     self.w_initialized = True
276
277 def _update_weights(self, xi, target):
278     """Apply Adaline learning rule to update the weights"""
279     output = self.activation(self.net_input(xi))
280     error = (target - output)
281     self.w_ += self.eta * 2.0 * xi * (error)
282     self.b_ += self.eta * 2.0 * error
283     loss = error**2
284     return loss
```

SGD with Feature Scaling

```
286 def net_input(self, X):
287     """Calculate net input"""
288     return X @ self.w_ + self.b_
289
290 def activation(self, X):
291     """Compute linear activation"""
292     return X
293
294 def predict(self, X):
295     """Return class label after unit step"""
296     return (self.activation(self.net_input(X)) >= 0.5).astype(int)
```

(No change, see p. 55)

```
254 ✓ def partial_fit(self, X, y):
255     """Fit training data without reinitializing the weights"""
256 ✓     if not self.w_initialized:
257         self._initialize_weights(X.shape[1])
258 ✓     if y.ravel().shape[0] > 1:
259 ✓         for xi, target in zip(X, y):
260             self._update_weights(xi, target)
261 ✓     else:
262         self._update_weights(X, y)
263     return self
```

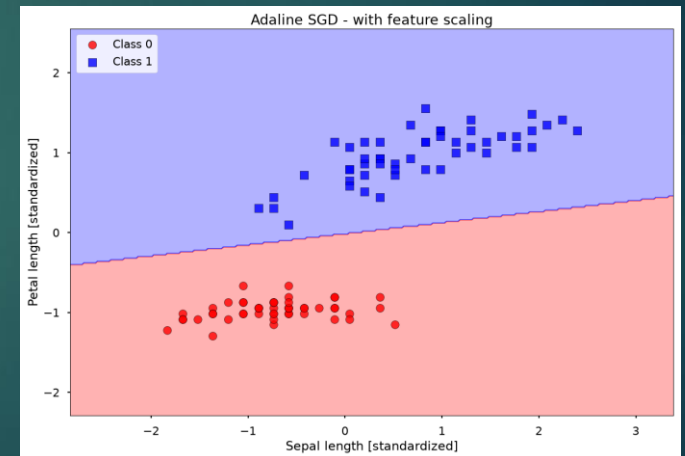
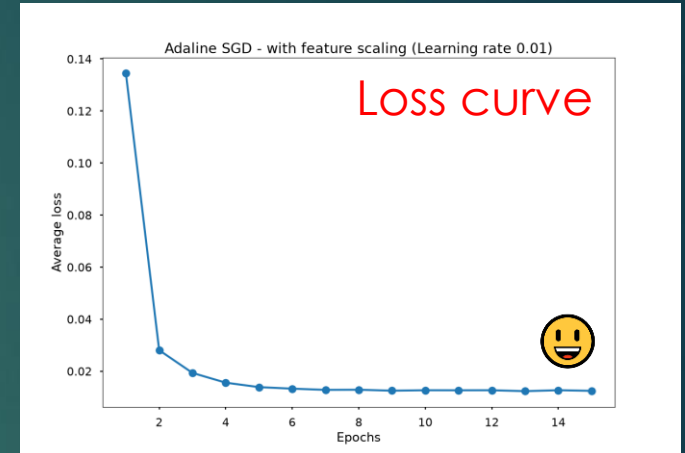
(Used for online learning)

```
ada_sgd.partial_fit(X_std[0, :], y[0])
```

SGD with Feature Scaling

```
298 ada_sgd = AdalineSGD(n_iter=15, eta=0.01, random_state=1)
299 ada_sgd.fit(X_std, y)
300 plt.plot(range(1, len(ada_sgd.losses_) + 1), ada_sgd.losses_, marker='o')
301 plt.title('Adaline SGD - with feature scaling (Learning rate 0.01)')
302 plt.xlabel('Epochs')
303 plt.ylabel('Average loss')
304 plt.show()

305
306 plot_decision_regions(X_std, y, classifier=ada_sgd)
307 plt.title('Adaline SGD - with feature scaling')
308 plt.xlabel('Sepal length [standardized]')
309 plt.ylabel('Petal length [standardized]')
310 plt.legend(loc='upper left')
311 plt.tight_layout()
312 plt.show()
```





Classwork

Assignment #1

不要使用 `scikit-learn` 函式庫完成本作業

- 目的：使用 **Adaline SGD 分類模型**，透過測試不同的特徵組合，觀察何種能力指標對科技公司篩選「優秀員工」最具鑑別度
- 應用情境：
 - ▶ 某科技公司記錄了 **300** 位雇員的「**四項**新進人員能力/條件」之檢定結果 (即**特徵**)
 - 語文能力 (浮點值 [0.0, 5.0])
 - 數理能力 (整數值 [0, 100])
 - 程式能力 (整數值 [1, 10])
 - 健康條件 (浮點值 [-1.0, 1.0])

} 依據檢定工具而有不同的成績分佈範圍
 - ▶ 以及各雇員在職一年後的**表現** (即**標籤**)
 - 0：普通
 - 1：優秀
- 資料集：**tech_employees.csv**

	Language_Skill	Math_Skill	Coding_Skill	Health_Condition	Performance_Label
0	1.87	95	4	-0.90	1
1	4.75	93	10	0.56	1
2	3.66	46	8	0.66	1
3	2.99	98	10	0.50	1
4	0.78	54	2	0.60	0
..	...	...	...	...	...
295	2.61	75	3	-0.76	0
296	3.85	23	3	0.46	0
297	1.08	27	1	-0.61	0
298	3.11	7	8	-0.77	1
299	0.43	91	6	-0.16	1

[300 rows x 5 columns]

Assignment #1

不要使用 scikit-learn 函式庫完成本作業

■ 實作任務與要求：

▶ 必須切分資料集

- 訓練集：前 200 筆 samples，用於訓練 Adaline SGD Model
- 測試集：後 100 筆 samples，用於測試以計算分類正確率 (Accuracy)
 - 分類正確 (即預測結果與真實標籤相符) 的筆數 ÷ 測試集樣本總筆數 (即 100)

▶ 特徵組合：

- 每次只使用 2 種特徵值 (例如：語文能力 & 數理能力)，總共會有 6 種組合
- 使用所有特徵值

▶ 使用各種訓練技巧：

- 特徵縮放 (Standardization)、資料隨機打散 (Shuffle)、嘗試不同的學習率或導入自適應學習率 (see p. 19)、嘗試不同的訓練輪次 (Epochs)，盡可能地得到最佳化訓練結果

特徵縮放中使用的統計值僅能以訓練集計算 (例如標準化所需要的平均值和標準差)，確保測試集是完全未看過的數據

Assignment #1

不要使用 `scikit-learn` 函式庫完成本作業

■ 預期結果 1 注意：測試集需要使用「以訓練集計算的平均值和標準差」進行一致的特徵縮放

► 輸出 (6 + 1) 種特徵組合訓練之模型得到的測試正確率比較結論

● 請依正確率高低排序呈現

Feature Combination	Test Accuracy
Math_Skill & Coding_Skill	0.87
All Four Features	0.87
Language_Skill & Coding_Skill	0.77
Coding_Skill & Health_Condition	0.76
Math_Skill & Health_Condition	0.64
Language_Skill & Math_Skill	0.61
Language_Skill & Health_Condition	0.53

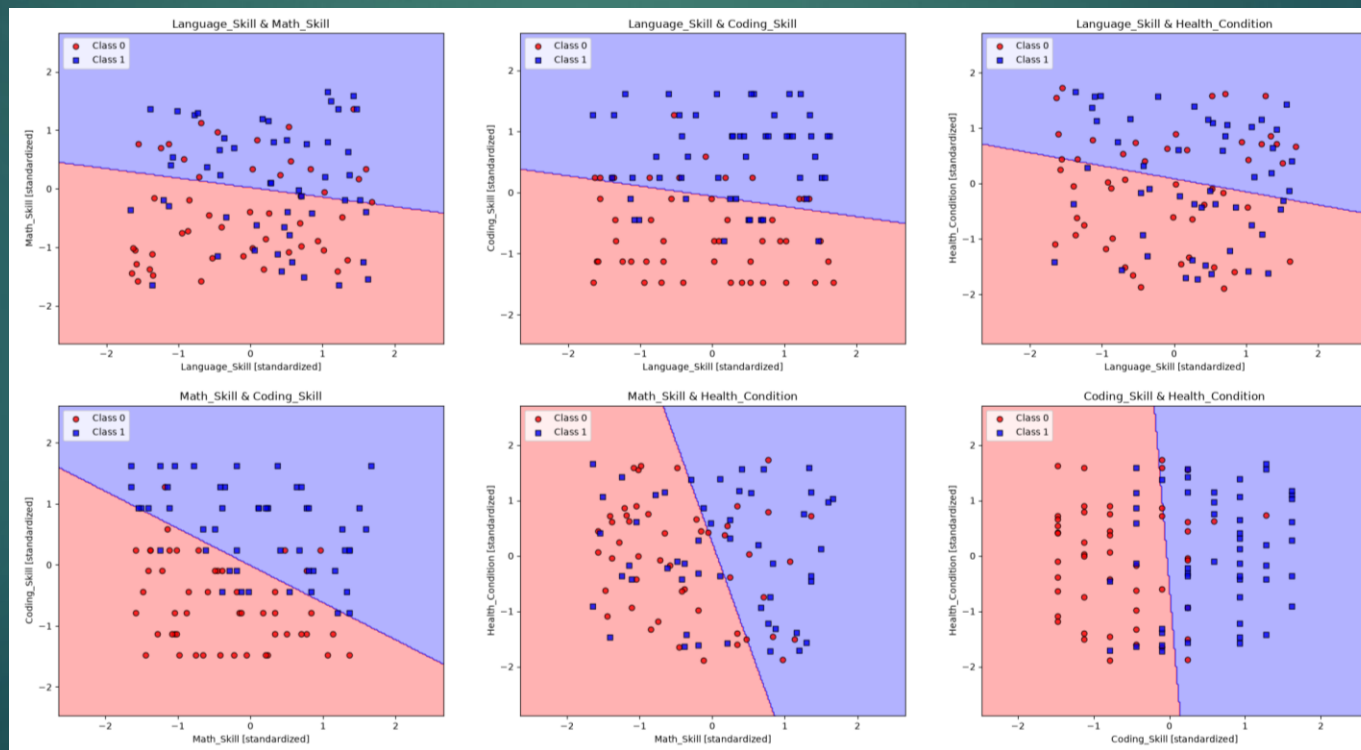
程式能力 (Coding_Skill) 是其中的首要指標
數理能力 (Math_Skill) 次之

Assignment #1

不要使用 scikit-learn 函式庫完成本作業

■ 預期結果 2

- ▶ 針對只使用 2 種特徵值之模型訓練結果，視覺化呈現決策邊界區域與測試集樣本的分佈





Thank you